

Notatki z wykładu: Algorytmy zachłanne

Problem wyboru zajęć (Activity Selection Problem)

1 kwietnia 2026

1 Wstęp do algorytmów zachłannych

Pomysł: Algorytm w każdym kroku dokonuje **lokalnie najlepszego wyboru** (wybiera element, który w danym momencie wydaje się najkorzystniejszy) [1, 7]. Celem jest, aby seria takich wyborów doprowadziła do rozwiązania optymalnego globalnie [7, 8]. Choć strategia ta nie zawsze działa, jest bardzo efektywna dla określonych problemów o strukturze zachłannej [8, 9].

2 Problem wyboru zajęć

2.1 Definicja problemu

Mamy zbiór n zajęć $S = \{z_1, z_2, \dots, z_n\}$ wymagających dostępu do tego samego, niepodzielnego zasobu (np. sala lekcyjna, procesor) [1, 8]. Każde zajęcie z_i charakteryzują [1, 9]:

- s_i – czas rozpoczęcia zajęcia,
- t_i – czas zakończenia zajęcia ($s_i < t_i$).

Zajęcia z_i oraz z_j nazywamy **zgodnymi**, jeśli ich przedziały czasowe są rozłączne: $(s_i, t_i) \cap (s_j, t_j) = \emptyset$ [1, 9].

Cel: Znaleźć podzbiór $A \subseteq S$ o **największej liczności** (maksymalnej liczbie zajęć), składający się z zajęć parami zgodnych [1, 9].

2.2 Przykład z wykładu

Dane dla $n = 6$ zajęć przedstawione na tablicy [1, 5, 10]:

i	1	2	3	4	5	6
s_i	1	0	2	4	7	7
t_i	3	4	6	7	9	10

Wstępną fazą algorytmu jest **sortowanie zajęć** w porządku niemalejącym według czasu zakończenia: $t_1 \leq t_2 \leq \dots \leq t_n$. Zajmuje to czas $O(n \log n)$ [1, 3, 10].

3 Podejście 1: Programowanie dynamiczne

3.1 Definicja podproblemu

Definiujemy S_{ij} jako zbiór zajęć zaczynających się po zakończeniu z_i i kończących przed rozpoczęciem z_j [2, 11]:

$$S_{ij} = \{z_k \in S : t_i \leq s_k < t_k \leq s_j\}$$

Dodajemy dwa sztuczne zajęcia: z_0 ($t_0 = 0$) oraz z_{n+1} ($s_{n+1} = \infty$). Wtedy cały problem $S = S_{0,n+1}$ [2, 11].

3.2 Własność optymalnej podstruktury i rekurencja

Jeśli z_k należy do rozwiązania optymalnego A_{ij} , to $A_{ij} = A_{ik} \cup \{z_k\} \cup A_{kj}$, gdzie A_{ik} i A_{kj} są rozwiązaniami optymalnymi dla mniejszych podproblemów [2, 12]. Niech $c[i, j] = |A_{ij}|$. Wzór rekurencyjny to [3, 13]:

$$c[i, j] = \begin{cases} 0 & \text{dla } i \geq j \text{ lub } S_{ij} = \emptyset \\ \max_{i < k < j, z_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{w przeciwnym razie} \end{cases}$$

Liczba podproblemów wynosi $\Theta(n^2)$, a koszt jednego to $O(n)$, co daje całkowitą złożoność $\Theta(n^3)$ [3, 13, 14].

4 Podejście 2: Algorytm zachłanny

Algorytm zachłanny zawsze wybiera zajęcie kończące się najwcześniej, co pozostawia najwięcej miejsca dla przyszłych zajęć [4, 15, 16].

Lemat 1 (O wyborze zachłannym). *Niech $z_m \in S_{ij}$ będzie zajęciem o najwcześniejszym czasie zakończenia ($t_m = \min\{t_k : z_k \in S_{ij}\}$). Wtedy [4, 15]:*

1. z_m należy do pewnego optymalnego rozwiązania dla S_{ij} .
2. Podproblem $S_{im} = \emptyset$. Wybór z_m redukuje problem do jednego podproblemu S_{mj} .

4.1 Pseudokod algorytmu

Algorithm 1 Activity-Selection(s, t)

```
 $n \leftarrow \text{length}(s)$ 
 $A \leftarrow \{z_1\}$  {Wybieramy zajęcie o najwcześniejszym końcu}
 $j \leftarrow 1$  {Indeks ostatnio dodanego zajęcia}
for  $i \leftarrow 2$  to  $n$  do
  if  $s_i \geq t_j$  then
     $A \leftarrow A \cup \{z_i\}$ 
     $j \leftarrow i$ 
  end if
end for
return  $A$ 
```

Dla przykładu z tablicy algorytm wybierze $A = \{z_1, z_4, z_5\}$ [5, 17].

5 Analiza poprawności (Dowód indukcyjny)

Pokażemy, że zbiór A wyznaczony przez algorytm jest optymalny [6, 18].

- **Baza:** Dla $n = 1$ algorytm wybiera z_1 , co jest oczywiście optymalne [6, 18].
- **Krok indukcyjny ($n > 1$):** Z Lematu wiemy, że istnieje optymalne rozwiązanie A_1 zawierające z_1 . Rozważmy podproblem $S' = \{z_k \in S : s_k \geq t_1\}$ [6, 19].
- Zbiór $A_1 \setminus \{z_1\}$ musi być rozwiązaniem optymalnym dla S' . Gdyby istniało lepsze rozwiązanie B dla S' ($|B| > |A_1 \setminus \{z_1\}|$), to zbiór $\{z_1\} \cup B$ byłby lepszy dla S niż A_1 , co przeczy optymalności A_1 [6, 19].
- Z założenia indukcyjnego nasz algorytm znajduje rozwiązanie optymalne dla mniejszego problemu S' . Łącząc z_1 z optymalnym rozwiązaniem dla S' , otrzymujemy rozwiązanie optymalne dla całego zbioru S [6, 20, 21].

6 Złożoność czasowa

- **Inicjalizacja i pętla:** $O(n)$ – przechodzimy przez zajęcia tylko raz [6, 22].
- **Sortowanie:** $O(n \log n)$ – faza wstępna [3, 22].
- **Całkowita złożoność:** $O(n \log n)$. Jest to znaczna poprawa względem $\Theta(n^3)$ z programowania dynamicznego [6, 23].